

Chapter 13 作业

学号：23336128

姓名：梁力航

问题1：关系存储策略的优缺点比较

在存储关系数据库时，有两种常见策略：

- a. 每个关系单独存储在一个文件中；
- b. 多个关系共享同一个文件。

分别写出这两种策略的两个优点和两个缺点。

回答：

a. 每个关系存储在一个文件中的优点：

- （1）物理组织简单清晰，目录结构与逻辑模式一一对应，便于管理、备份与恢复。
- （2）可以针对每个关系独立选择合适的文件组织和索引结构，便于物理调优（如聚簇、非聚簇索引等）。

a. 每个关系存储在一个文件中的缺点：

- （1）关系数量较多时会产生大量小文件，带来文件管理系统开销和元数据空间浪费。
- （2）无法在物理上把经常一起访问的不同关系聚集在一起，跨表连接时局部性较差，需要更多块访问。

b. 多个关系存储在一个文件中的优点：

- （1）可以将语义上相关、经常联合访问的关系物理上聚集在同一文件甚至同一块内，提高缓存命中率和I/O局部性。

- (2) 减少操作系统需管理的文件数目，小关系可以共享块，降低文件头和未用空间的浪费。

b. 多个关系存储在一个文件中的缺点：

- (1) 记录格式更复杂，访问某个关系时必须根据额外标识过滤记录，增加解析开销，实现难度大。
- (2) 并发控制、备份和恢复更难做到精细化（很难只锁定或备份单个关系），也不利于对单个关系进行独立物理调优。

问题2：可变长度记录与 null bit map

在可变长度记录表示中，使用 null bit map 来指示属性是否具有空值。

- a. 对于变长字段，如果其值为 NULL，偏移（offset）和长度（length）字段将存储什么？
- b. 在一些应用中，元组具有大量属性，其中大多数属性为 NULL。能否修改记录表示，使得 NULL 属性的唯一开销是 bit map 中的一个 bit？

回答：

a. 变长字段为 NULL 时 offset 与 length 的处理：

当某个变长属性取值为 NULL 时，其实际数据并不会出现在变长数据区中。由于是否为 NULL 已由 null bit map 唯一决定，记录格式中可以将该属性的 **length** 设为 0，而 **offset** 保持为上一个非空变长字段结束后的位置（或设为任意约定的值并在实现中忽略）。访问时首先检查 null bit map，若为 NULL 则不再根据 offset/length 去读取数据。

b. 仅以 bit map 作为 NULL 属性开销的一种改进方案：

可以将记录划分为三个部分：定长字段区、null bit map 以及紧凑的变长数据区，**只**为**非空的变长属性保存 (offset, length) 信息**，并按属性序号顺序连续存放其值。对于 NULL 属性，不再单独保留 offset/length 条目，也不在变长数据区中占据任何空间，其是否存在完全由 bit map 中的 1 bit 指示。查找第 i 个变长属性时，通过统计 bit map 中前 i 个属性中非空位的个数，定位到对应的 offset/length 项和数据。这样 NULL 属性的额外开销就只有 bit map 中的 1 bit。

问题3：记录到块的分配对性能的影响

解释为什么为 records 分配 block 的方式会显著影响数据库系统的性能。

回答：

磁盘 I/O 的基本单位是块（page），CPU 时间通常远小于磁盘访问时间，因此访问一个查询所需的块数 直接决定了整体性能。若记录在块上的分配不合理，例如相关记录分散在许多块中、同一块中混入大量与查询无关的记录，将导致查询需要访问更多磁盘块，增加随机 I/O 次数。

相反，如果将经常一起访问的记录（如同一关系的某个范围、或同一聚簇键上的记录）放在少量连续块中，顺序扫描或索引范围查询便可在很少的块内完成，大幅减少磁盘寻道与旋转延迟。因此，records 到 blocks 的物理分配策略（聚簇、填充因子、顺序组织等）会显著影响数据库系统的总体性能。

问题4：顺序文件中的溢出块

在顺序文件组织中，即使当前只有一个溢出记录，也往往要使用一个单独的溢出块，解释原因。

回答：

顺序文件要求主数据块中的记录按照某个排序码有序存放。若直接把新插入的记录放入已满的主块中，必须移动大量记录甚至重写后续多个块才能保持顺序，这在频繁插入的场景下代价极高。

因此，一旦某个主块装满，后续插入（即便当前只有一条）也被放入与该块关联的溢出块中，只需维护主块到溢出块的指针即可，既保持主块顺序不变，又为将来更多溢出记录预留空间。这样可以显著降低插入时的重组织开销，并保持顺序扫描和范围查询的实现相对简单。

问题5：section 与 takes 的多表聚集文件结构

考虑关系 **section** 与 **takes**。给出这两个关系的一个示例实例，其中包含 3 个 **section**，每个 **section** 有 5 名学生；并为这两个关系设计一个使用 **多表聚集** (clustered file organization) 的文件结构。

回答：

示例关系实例（仅示意主要属性）：

```
section(course_id, sec_id, semester, year, building, room)
```

```
-----  
( 'CS101', '1', 'Fall', 2024, 'A', '101' )
```

```
( 'CS102', '1', 'Fall', 2024, 'A', '102' )
```

```
( 'CS103', '1', 'Fall', 2024, 'B', '201' )
```

```
takes(id, course_id, sec_id, semester, year, grade)
```

```
-----  
每个 section 对应 5 条 takes 记录，例如：
```

```
id: s1~s5   选 CS101-1, Fall 2024
```

```
id: s6~s10  选 CS102-1, Fall 2024
```

```
id: s11~s15 选 CS103-1, Fall 2024
```

多表聚集文件结构设计：

采用以 **(course_id, sec_id, semester, year)** 为聚集键的多表聚集文件。按该键排序后，将每个 **section** 的元组与其所有 **takes** 元组物理上聚集存放在一起：

- 在一个块（或若干连续块）中，首先存放某个 **section** 的记录，然后紧跟该 **section** 的 5 条 **takes** 记录。
- 若一个块放不下一个 **section** 及其所有 **takes**，可为该 **section** 分配一个块链（主块 + 若干从块），整条链只存放该 **section** 及其学生记录。
- 整个文件按聚集键顺序排列 **section** 块链：先是 CS101-1 的块链，再是 CS102-1，再是 CS103-1，以此类推。

这种组织方式保证了：给定一个 **section**，可以在很少的块访问内读出其全部学生选课信息；对某学期的所有 **section** 进行顺序扫描时，也能顺带高效获取对应的 **takes** 记录，充分利用块 I/O 的局部性。